

AVL Trees

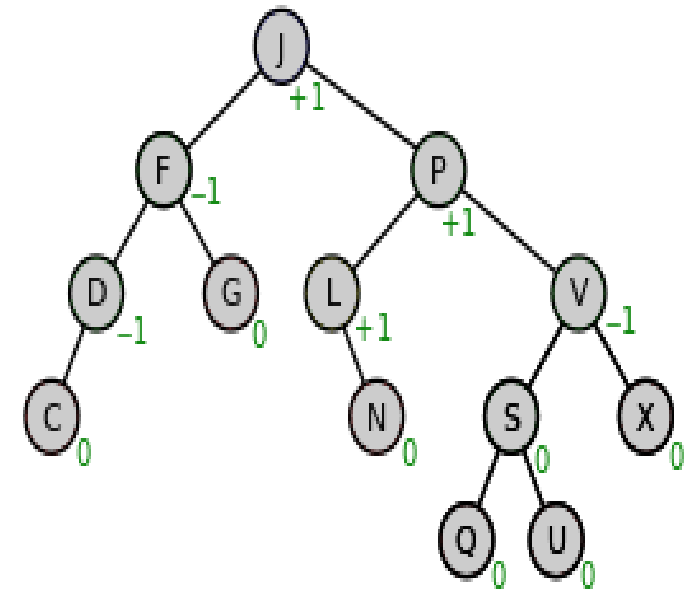
Adelson- Velsky - Landis

AVL Tree

AVL tree is a self-balancing binary search tree.

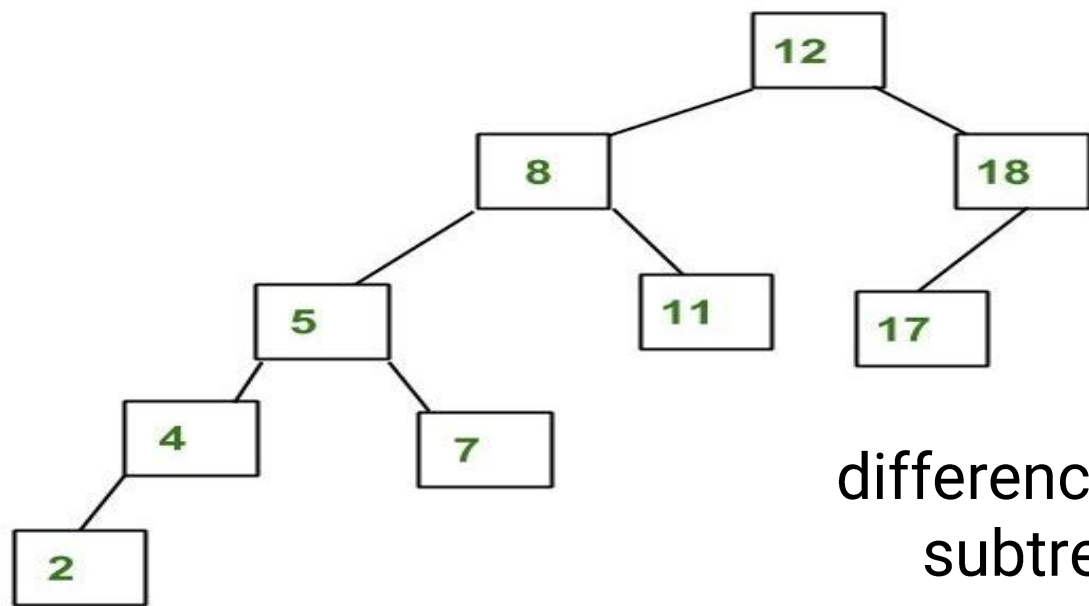
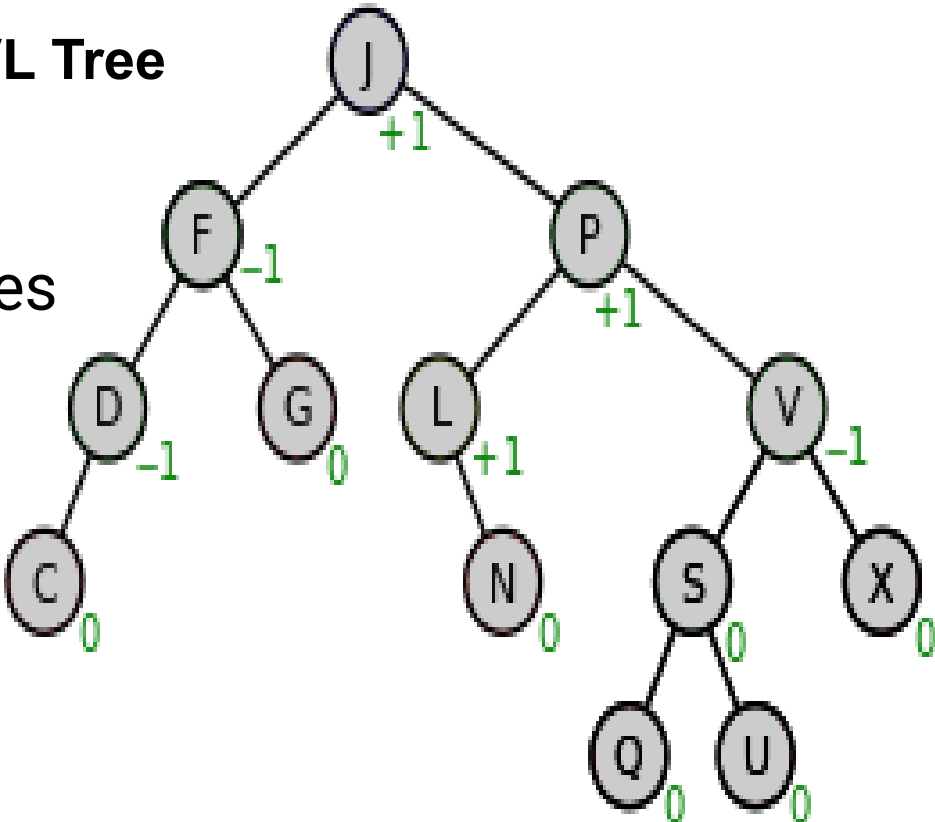
In an AVL tree

- the heights of the two child subtrees of any node differ by at most one;
- if heights differ by more than one, rebalancing is done to restore this property.



Differences between heights of left and right subtrees for every node is less than or equal to 1

AVL Tree



Tree is not AVL
differences between heights of left and right subtrees for 8 and 12 is greater than 1

Why AVL Trees?

Most of the BST operations (e.g., search, max, min, insert, delete.. etc) take **$O(h)$** time where h is the height of the BST.

The cost of these operations may become $O(n)$ for a skewed Binary tree.

For Balanced tree

- ❑ The height of the tree remains $O(\text{Log}n)$.
- ❑ After every insertion and deletion, we can guarantee an upper bound of $O(\text{Log}n)$

Thus, height of an AVL tree is always $O(\text{Log}n)$ where n is the number of nodes in the tree.

Insertion

To make sure that the given tree remains AVL after every insertion, we must augment the standard BST insert operation to perform some re-balancing.

Following are two basic operations that can be performed to re-balance a BST without violating the BST property (**keys(left) < key(root) < keys(right)**).

1) Left Rotation

2) Right Rotation

T1, T2 and T3 are subtrees of the tree rooted with y (on the left side) or x (on the right side)

 y
 / \
 x T3
 / \
T1 T2

Right Rotation

- - - - - >
< - - - - -

Left Rotation

 x
 / \
 T1 y
 / \
T2 T3

Keys in both of the above trees follow the following order

keys(T1) < key(x) < keys(T2) < key(y) < keys(T3)

So BST property is not violated anywhere.

Node insertion in AVL tree

Steps to follow for insertion

Let the newly inserted node be w

- 1) Perform standard BST insert for w .
- 2) Starting from w , travel up and find the first unbalanced node.
 - Let z be the first unbalanced node,
 - y be the child of z that comes on the path from w to z .
 - x be the grandchild of z that comes on the path from w to z
- 3) Re-balance the tree by performing appropriate rotations on the subtree rooted with z .

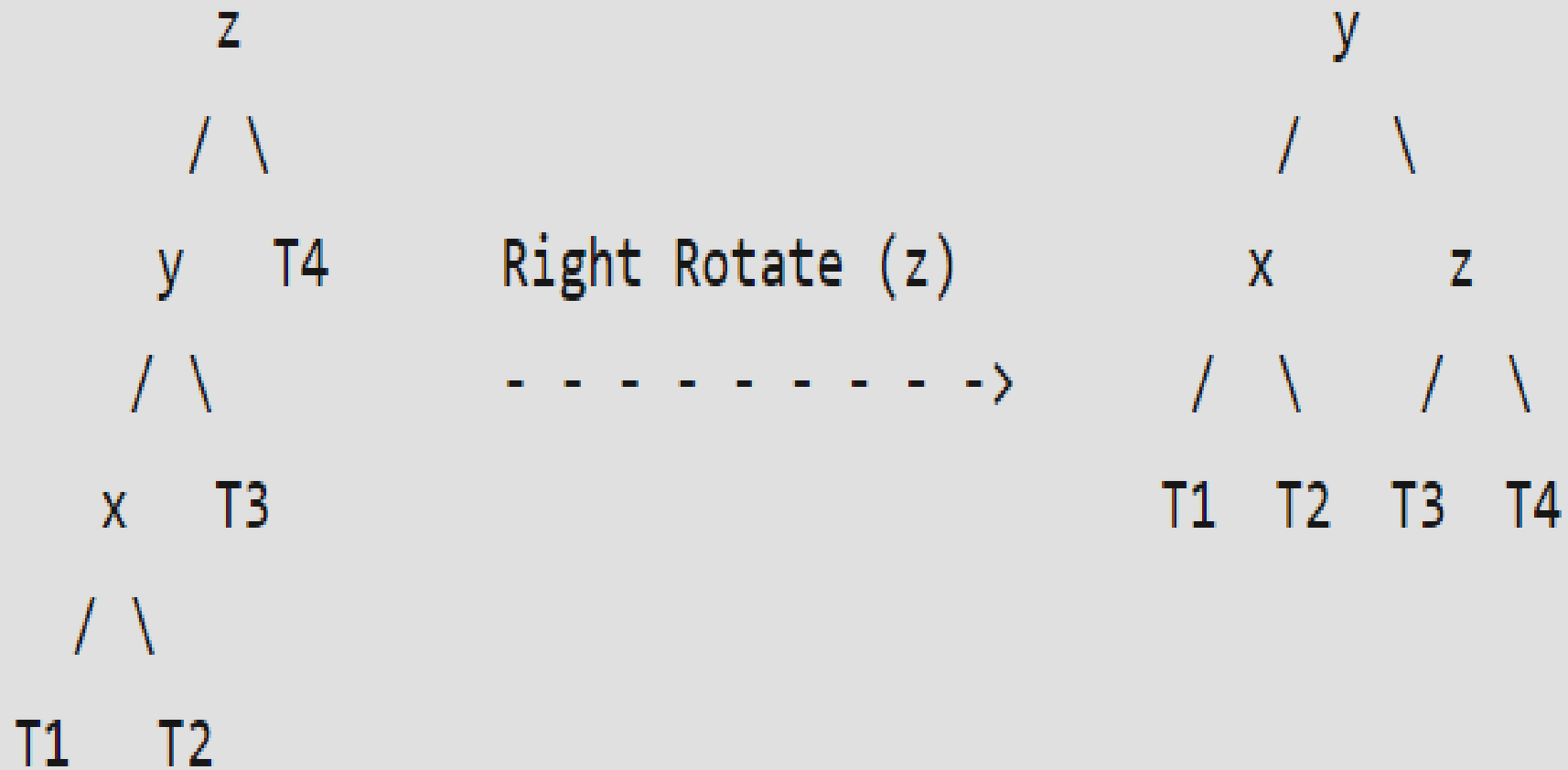
There can be 4 possible cases that needs to be handled as x , y and z can be arranged in 4 ways.

- a) y is left child of z and x is left child of y (Left Left Case)
- b) y is left child of z and x is right child of y (Left Right Case)
- c) y is right child of z and x is right child of y (Right Right Case)
- d) y is right child of z and x is left child of y (Right Left Case)

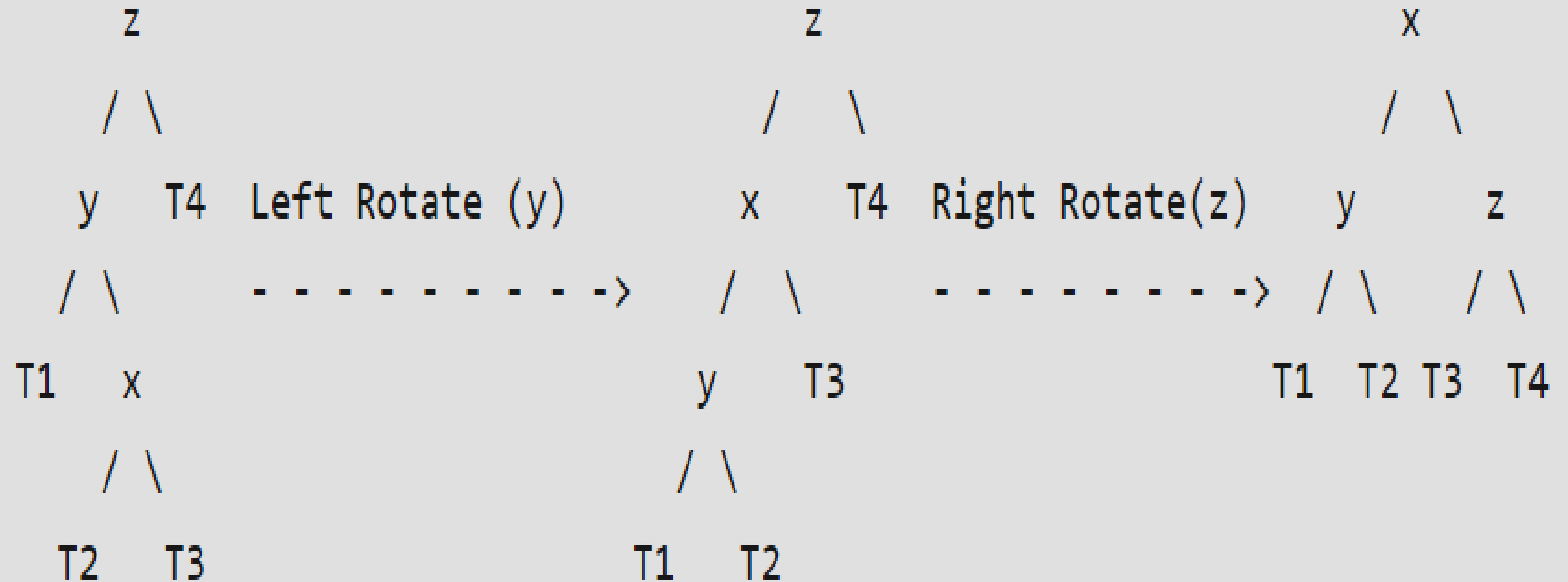
In all of the cases, we only need to re-balance the subtree rooted with z and the complete tree becomes balanced as the height of subtree (After appropriate rotations) rooted with z becomes same as it was before insertion.

a) Left Left Case

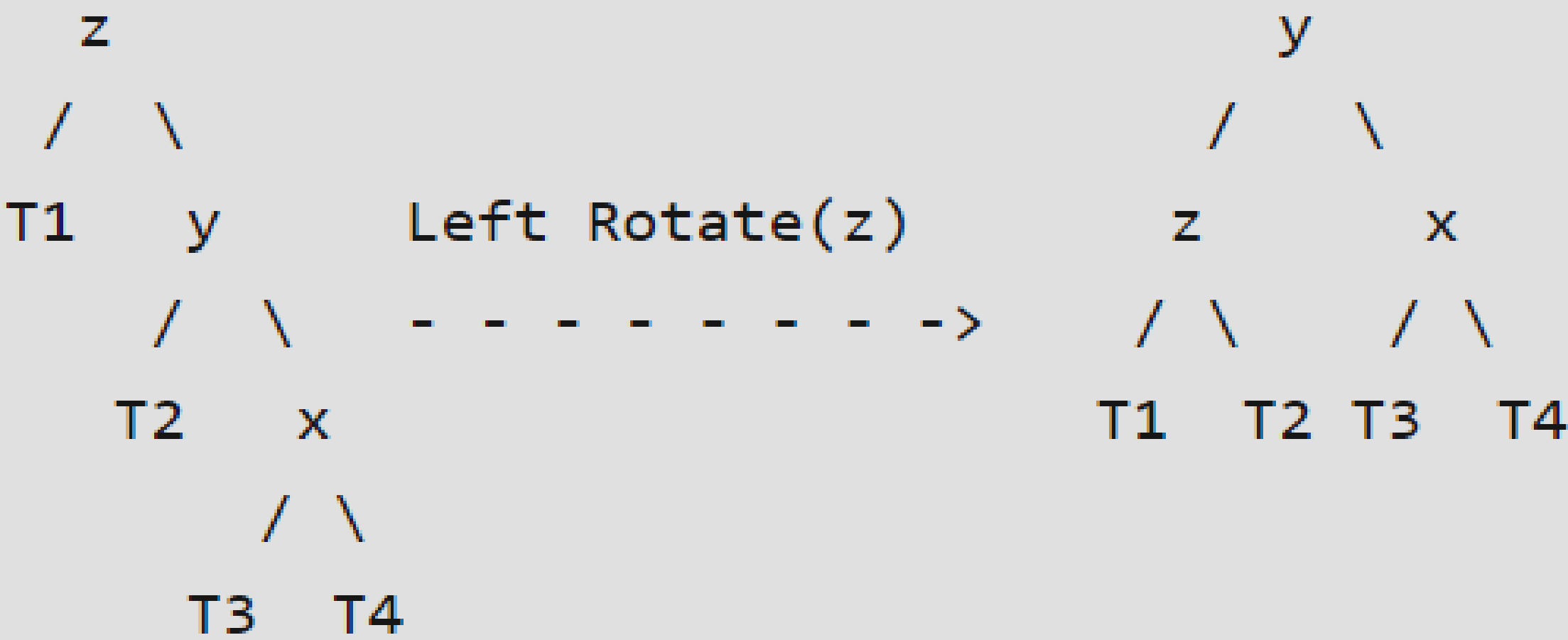
T1, T2, T3 and T4 are subtrees.



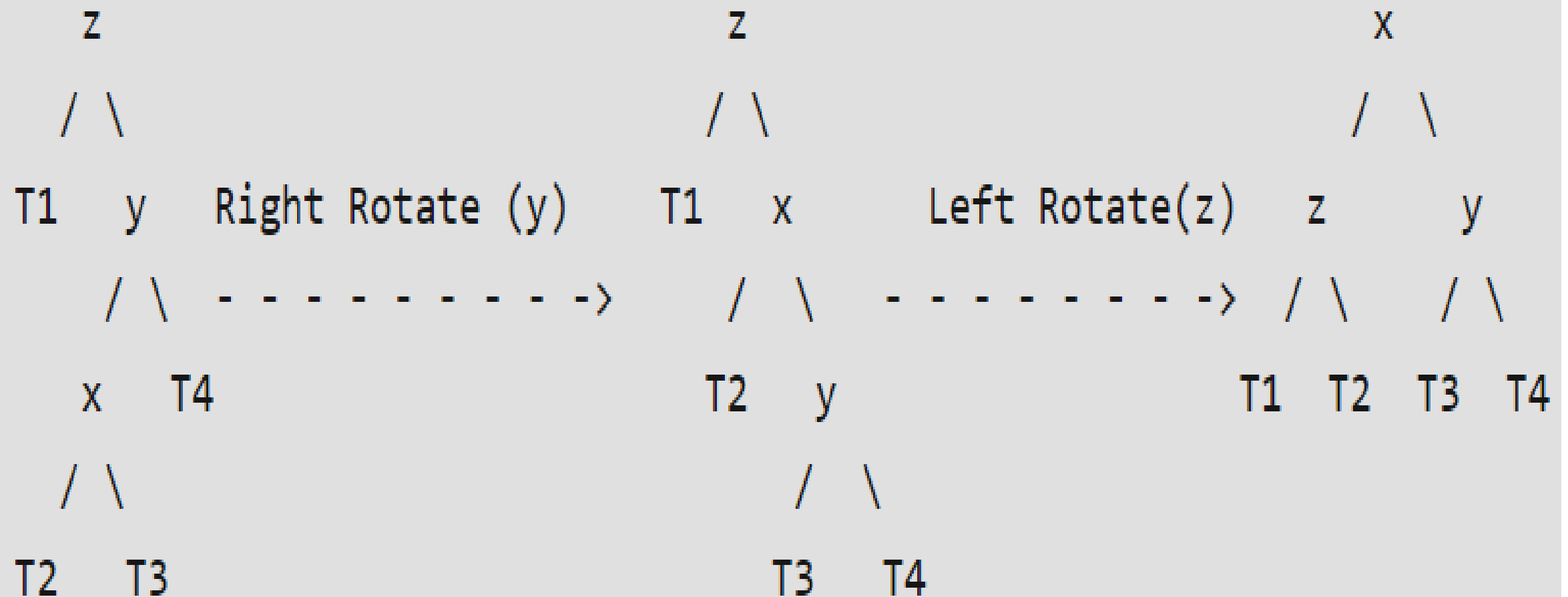
b) Left Right Case



c) Right Right Case



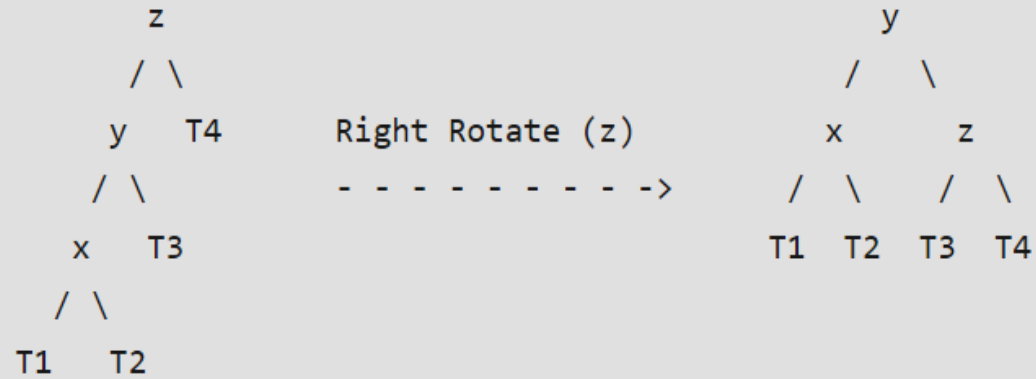
d) Right Left Case



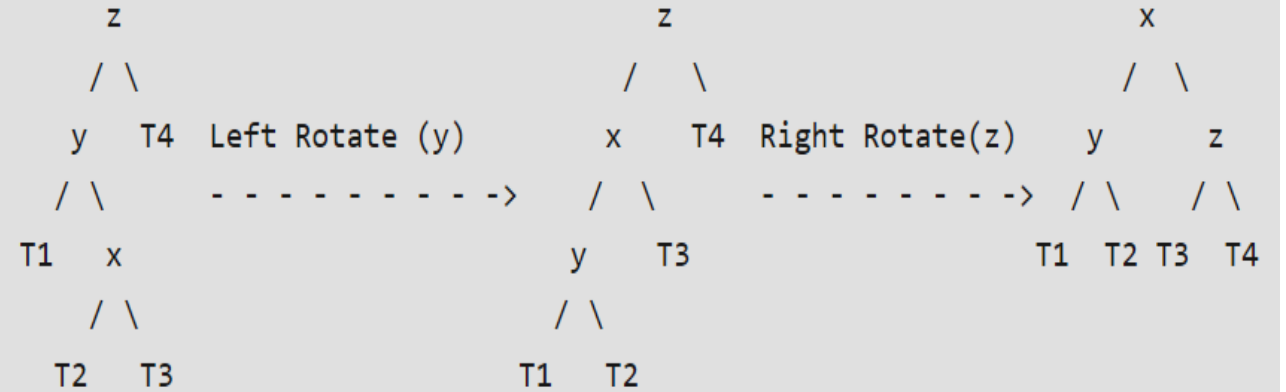
Following are the operations to be performed in above mentioned 4 cases

a) Left Left Case

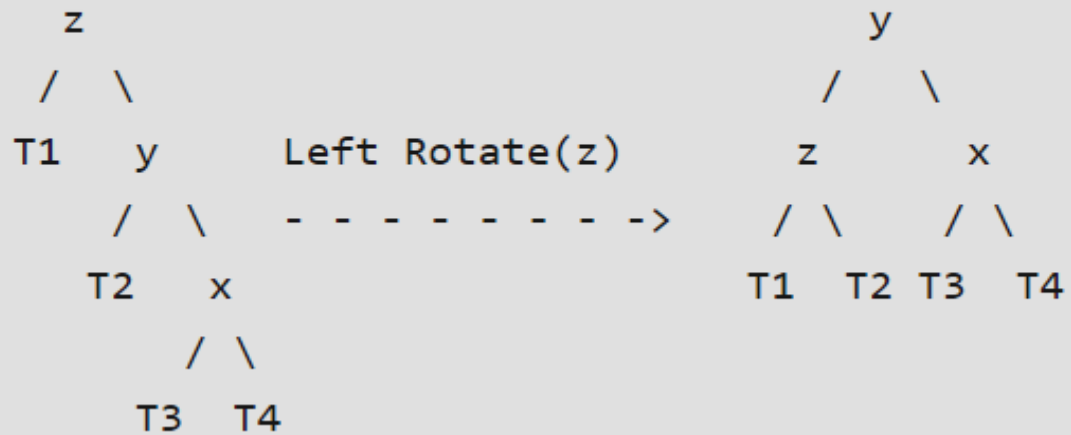
T1, T2, T3 and T4 are subtrees.



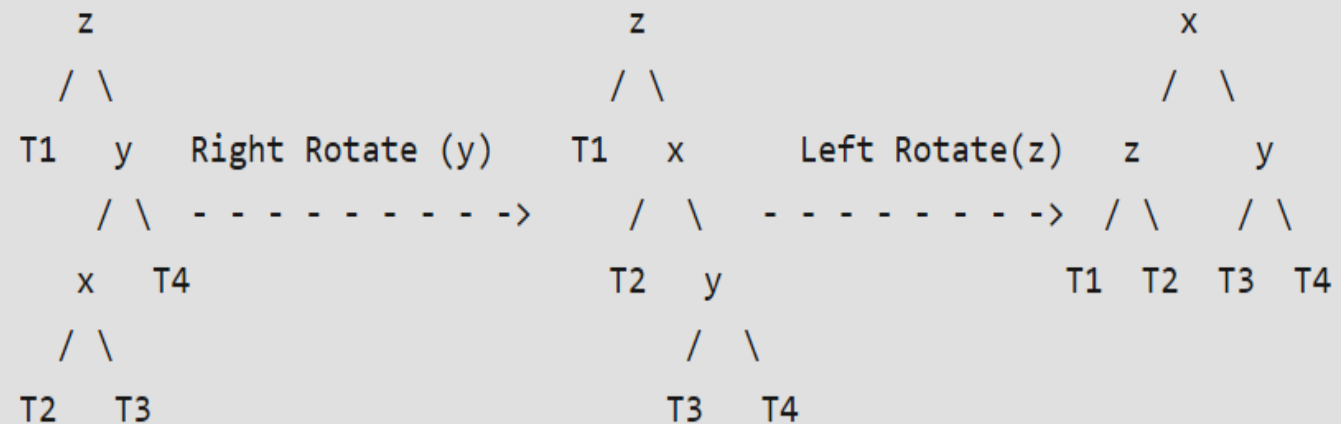
b) Left Right Case

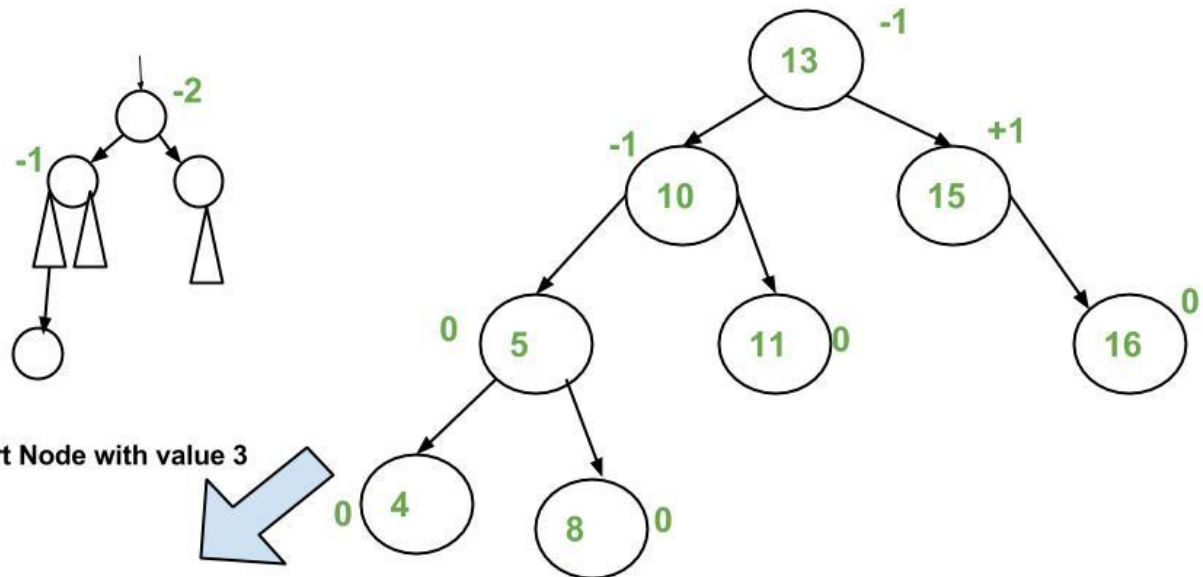


c) Right Right Case

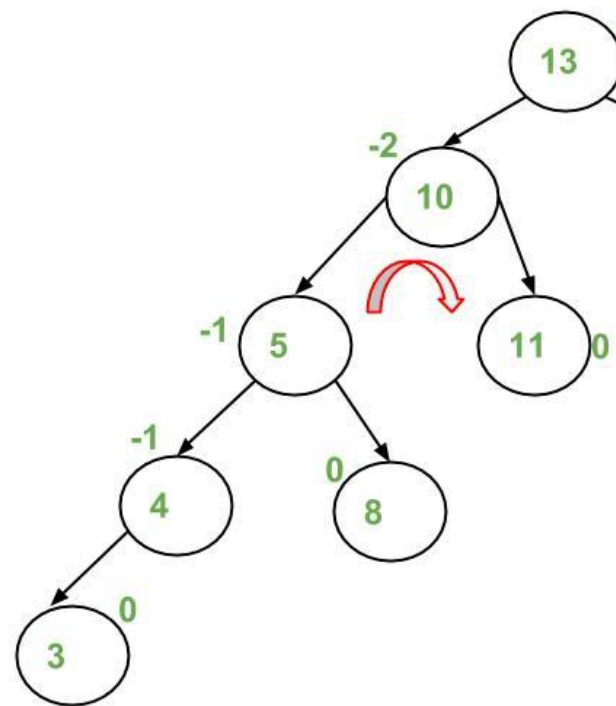


d) Right Left Case

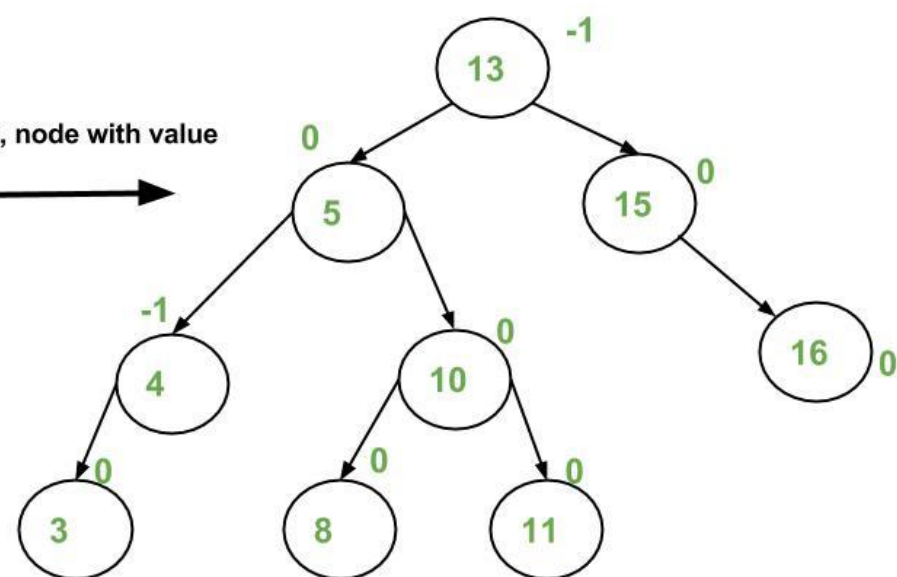




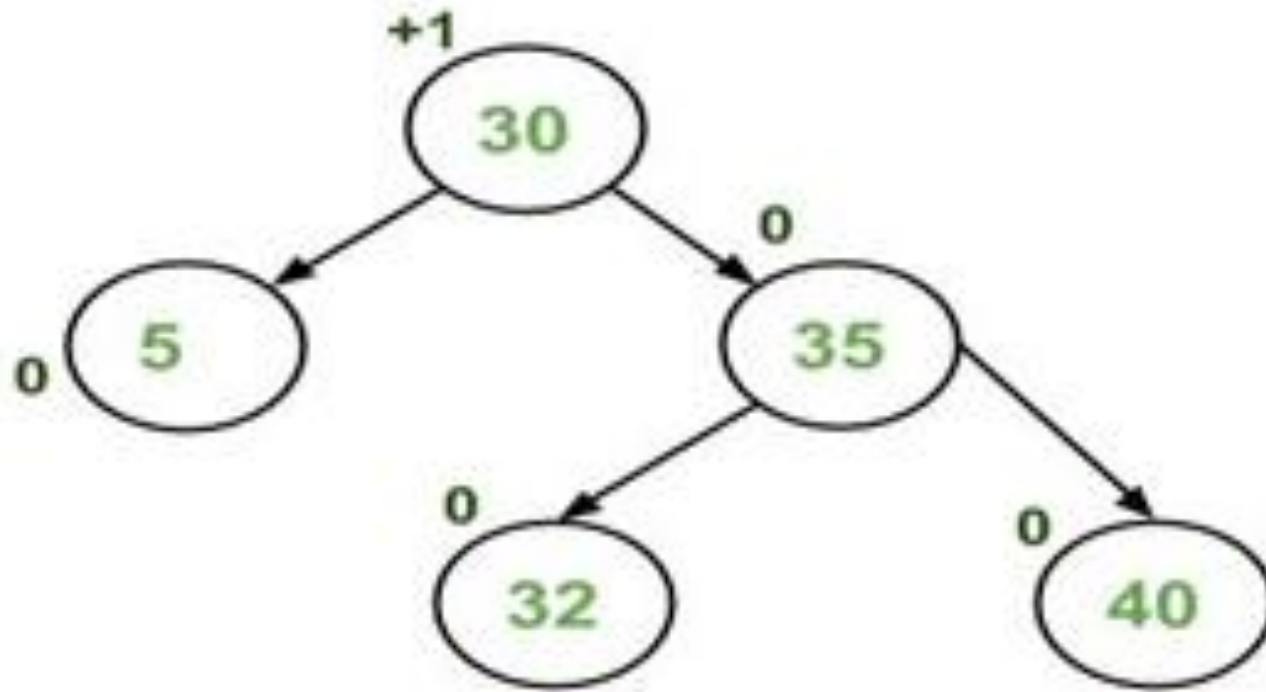
Insert Node with value 3

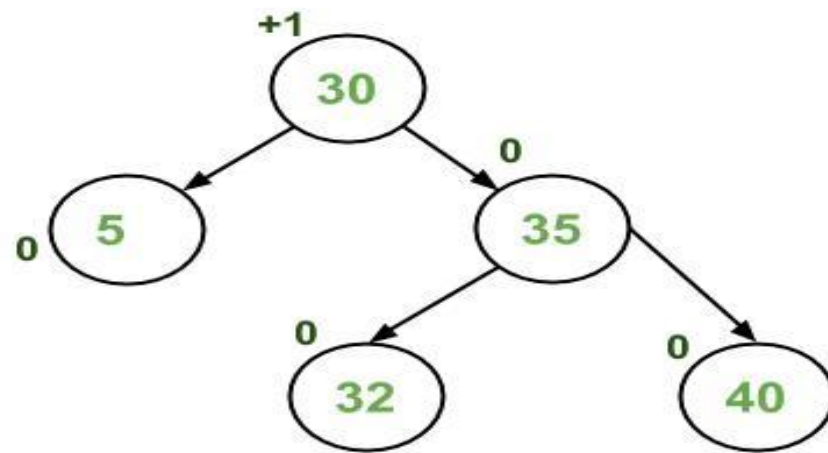
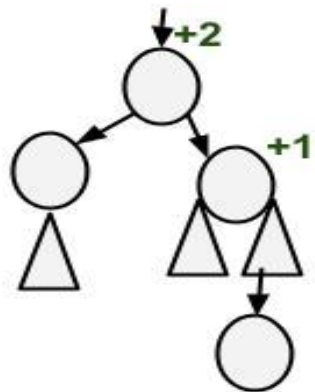


Rotating Right, node with value 10 as pivot

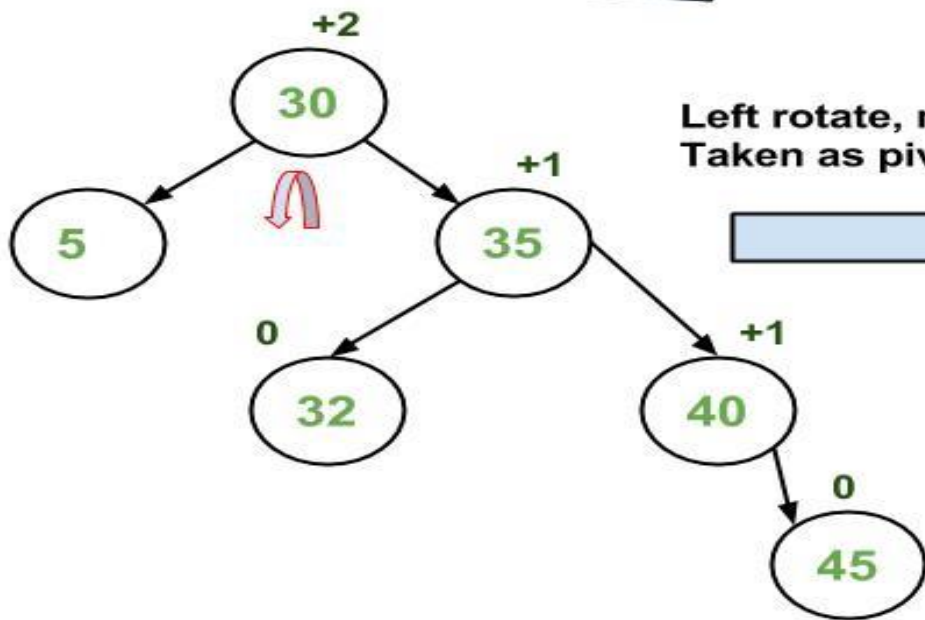


Insert 45

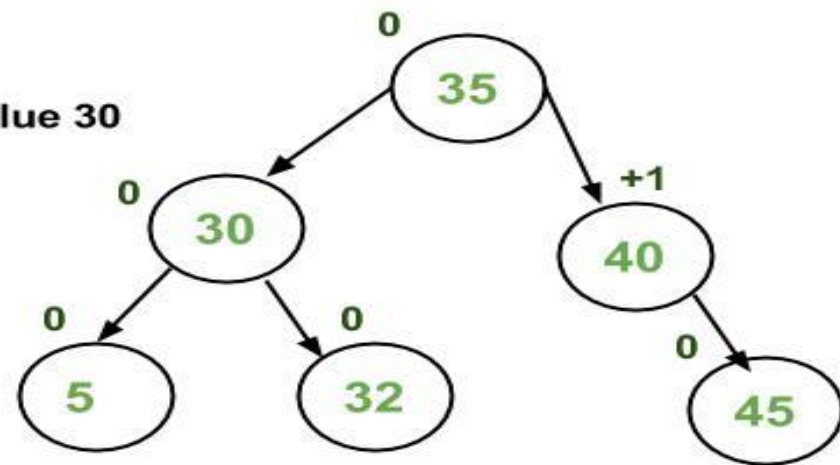




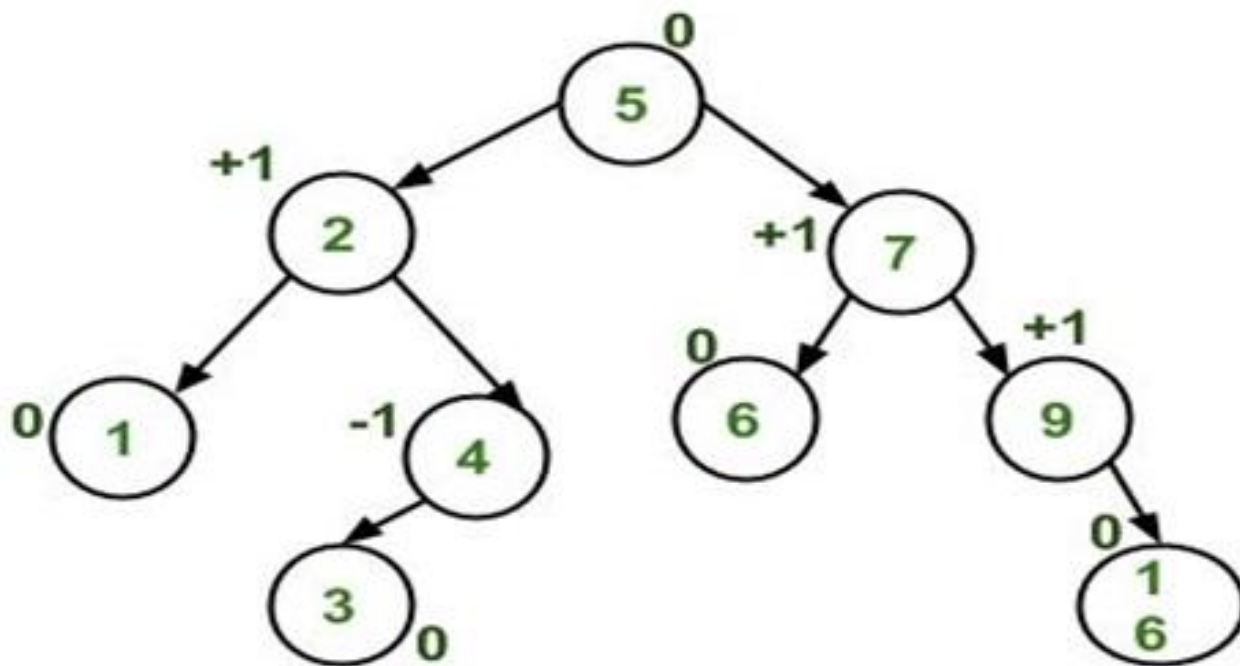
Insert 45

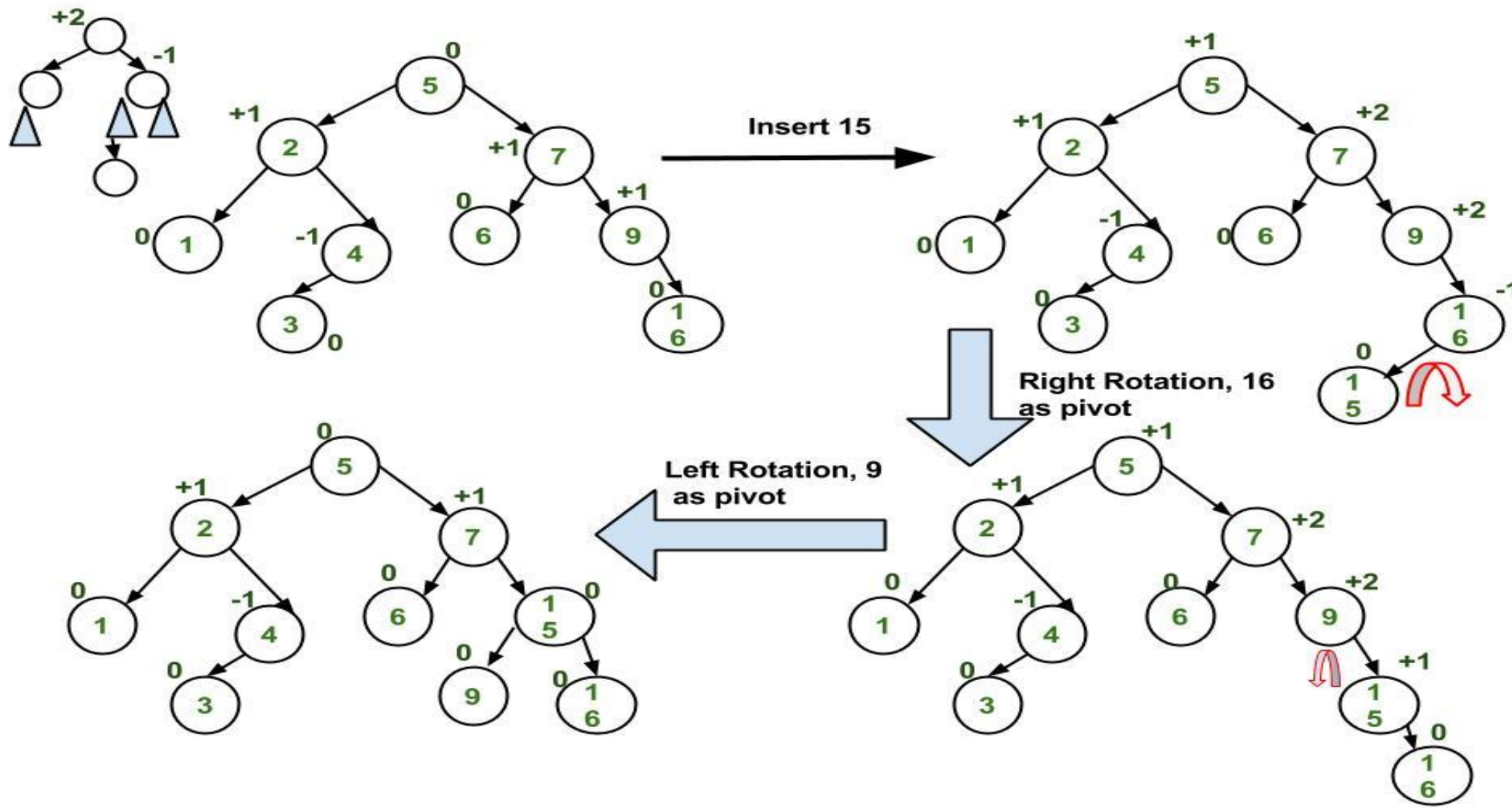


Left rotate, node with value 30
Taken as pivot



Insert 15





Insert following keys in an AVL tree
45, 32, 46, 12, 23,

Deletion in AVL Tree

- Deleting a node from an AVL tree is similar to that in a binary search tree.
- Deletion may disturb the balance factor of an AVL tree and therefore the tree needs to be rebalanced in order to maintain the AVLness.
- The two types of rotations are L rotation and R rotation

Let us consider that,

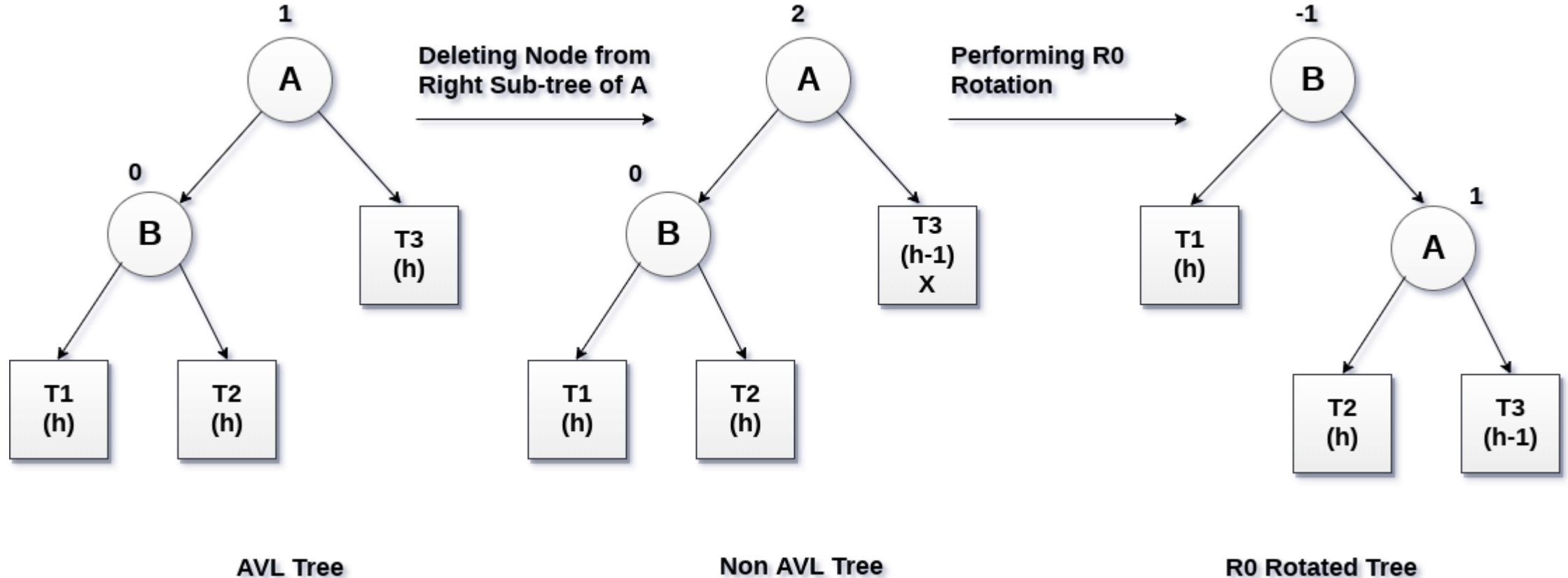
- A is the critical node and
- B is the root node of its left sub-tree.

If node X, present in the right sub-tree of A, is to be deleted, then there can be **three different situations**:

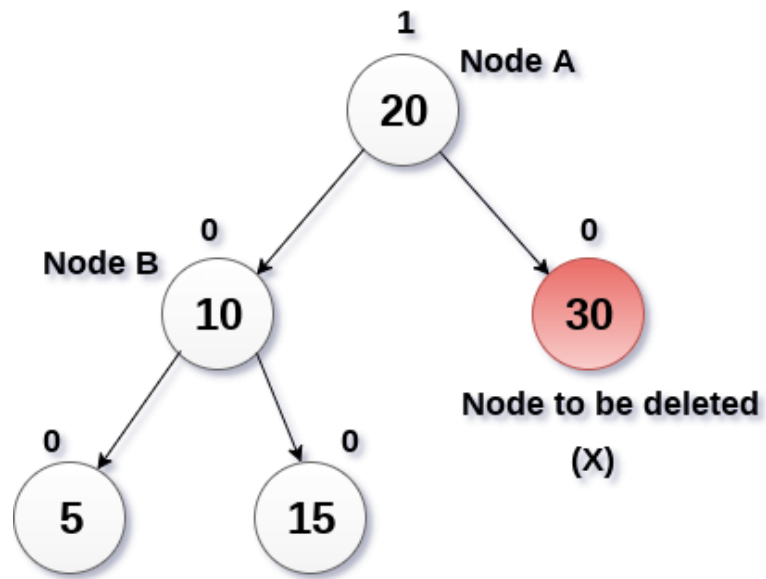
R0 rotation (Node B has balance factor 0)

If the node B has 0 balance factor, and the balance factor of node A disturbed upon deleting the node X, then the tree will be rebalanced by rotating tree using R0 rotation.

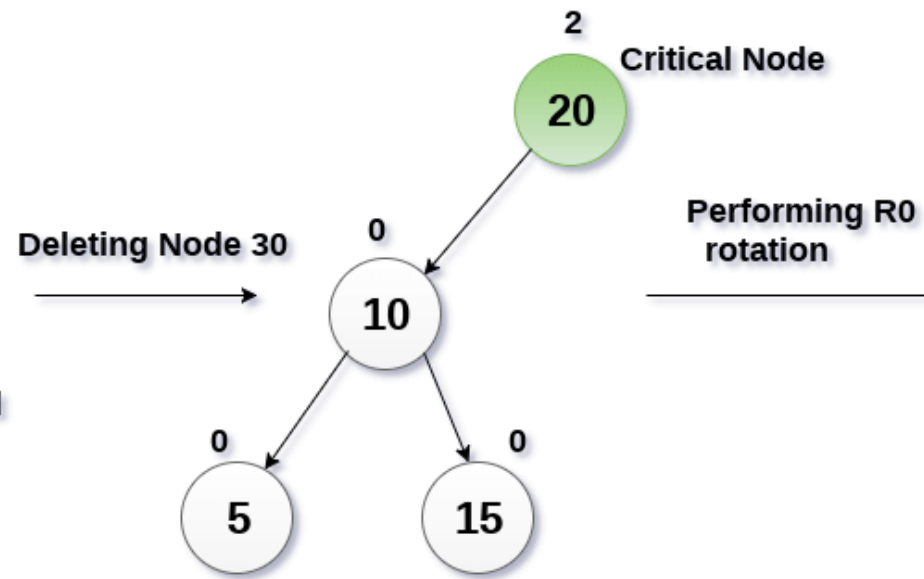
R0 → LL Case → Right Rotation at critical node



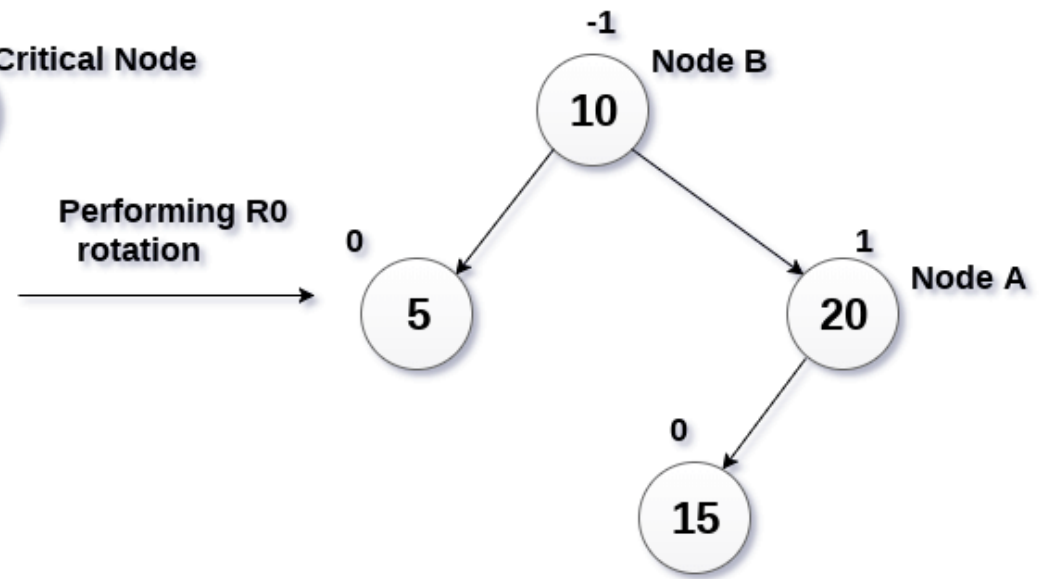
Example R 0



AVL Tree



Non AVL Tree



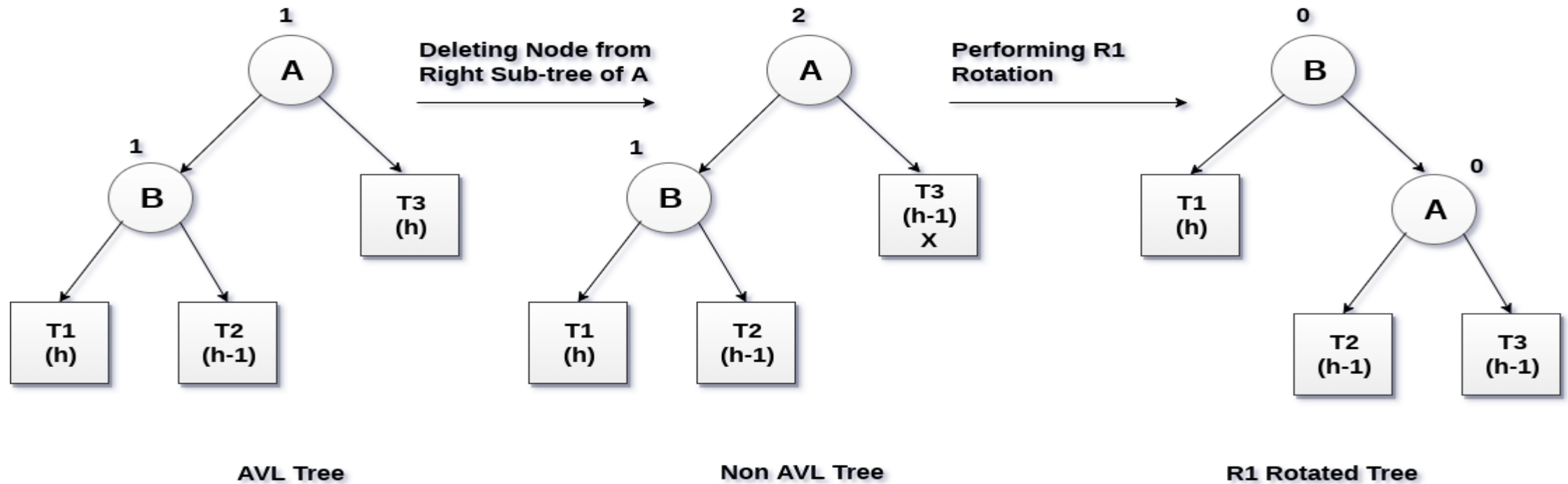
R0 Rotated Tree

R1 Rotation (Node B has balance factor 1)

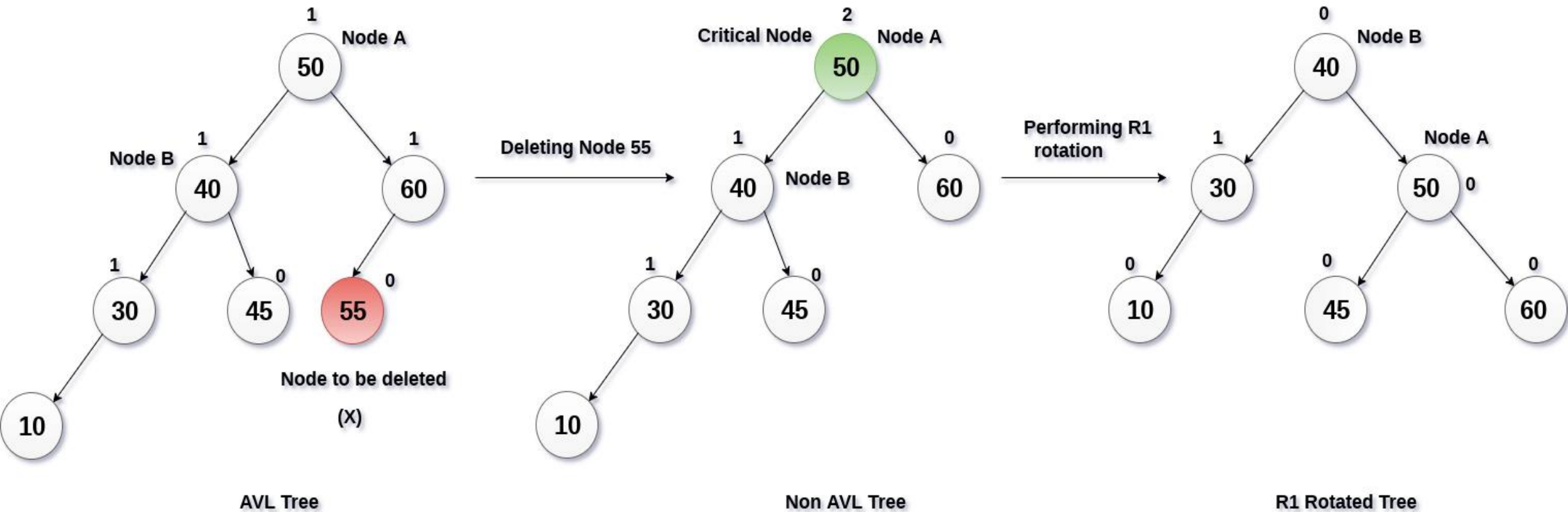
R1 Rotation is to be performed if the balance factor of Node B is 1.

In R1 rotation, the critical node A is moved to its right having sub-trees T2 and T3 as its left and right child respectively. T1 is to be placed as the left sub-tree of the node B.

R1 → LL Case → Right Rotation at critical node



Example R 1



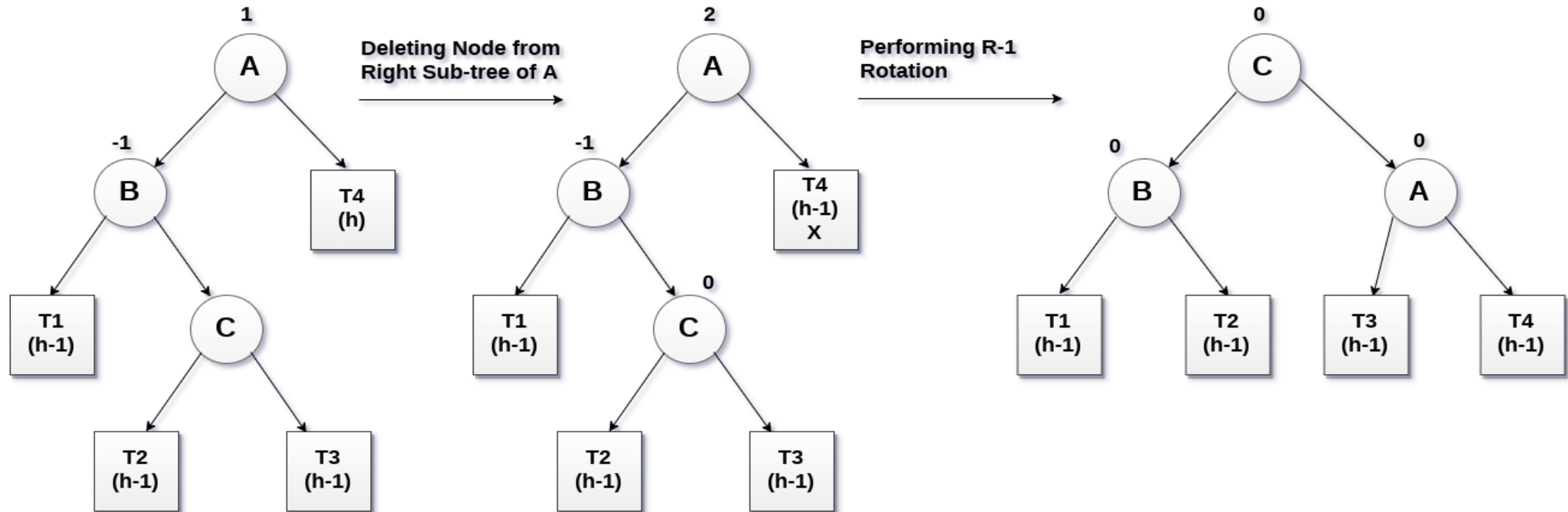
R -1 Rotation (Node B has balance factor -1)

R-1 rotation is to be performed if the node B has balance factor -1.

This case is treated in the same way as **LR rotation**.

In this case, the node C, which is the right child of node B, becomes the root node of the tree with B and A as its left and right children respectively.

R-1 → LR Case → Left Rotation at B and then Right Rotation at critical node(A)

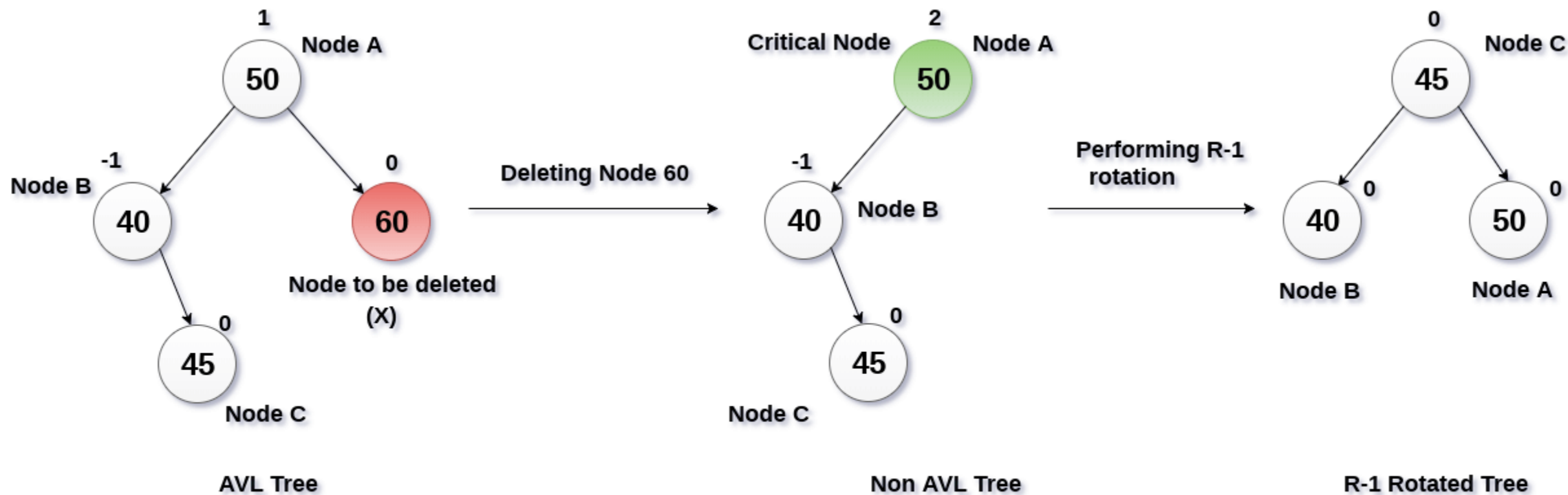


AVL Tree

Non AVL Tree

R-1 Rotated Tree

Example R-1



Summary

In deletion of any node in AVL tree, The two types of rotations can be made .
They are L rotation and R rotation.

As R rotations are discussed in previous slide. L rotations are the mirror images of them.

R rotations

R0 → LL Case

R1 → LL case

R -1 → LR case

L rotations

L0 → RR Case

L1 → RR Case

L -1 → RL Case

{Note – for rotation check previous Balance factor with sibling}

Rules of Rotation

IF using **L-R** for balancing Factor, then

R - Rotations

R0 = LL Case

R1 = LL case

R -1 = LR case

L - Rotations

L0 = RR Case

L1 = RL Case

L -1 = RR Case

IF using **R - L** for balancing Factor, then

R - Rotations

R0 = LL Case

R -1 = LL case

R 1 = LR case

L - Rotations

L0 = RR Case

L-1 = RL Case

L1 = RR Case

From Symmetry, we can derive other Rotation as well.

Note

R Rotations in L-R will be a mirror case of L rotation in R-L

Ex

if BF = L-R

R - Rotations

R0 = LL Case

R1 = LL case

R -1 = LR case

Then for BF in R-L will be

L0 = RR Case

L-1 = RL Case

L1 = RR Case

and likewise other cases can be derived easily

Construct AVL Tree

Insert

- 12, 19, 17, 6, 18, 13, 25, 36, 80

- Delete

17, 6, 18, 12

Construct AVL Tree

Insert

- 14, 11, 19, 7, 12, 17, 53, 4, 8, 13, 20, 60, 70

- Delete

8, 7, 11, 14, 17